

ITCEN Solution Frontend 아키텍처 가이드

목차

1. [프로젝트 개요](#)
 2. [기술 스택](#)
 3. [아키텍처 구조](#)
 4. [폴더 구조](#)
 5. [개발 환경 설정](#)
 6. [주요 기능별 구성](#)
 7. [상태 관리](#)
 8. [라우팅 시스템](#)
 9. [스타일링 시스템](#)
 10. [공통 컴포넌트](#)
 11. [멀티 탭 시스템](#)
 12. [빌드 및 배포](#)
 13. [개발 가이드라인](#)
 14. [성능 최적화](#)
 15. [트러블슈팅](#)
-

프로젝트 개요

ITCEN Solution Frontend는 React 18.2 기반의 현대적인 웹 애플리케이션으로, Domain-Driven Design(DDD) 아키텍처 패턴을 적용하여 확장 가능하고 유지보수가 용이한 구조로 설계되었습니다.

주요 특징

-  **현대적 기술 스택**: React 18.2, TypeScript, Vite
-  **일관된 디자인 시스템**: Material-UI v5 기반
-  **확장 가능한 아키텍처**: Domain-Driven Design 적용
-  **고성능**: Vite 빌드 도구, 코드 스플리팅
-  **개발자 경험**: TypeScript, ESLint, Prettier
-  **반응형 디자인**: 모바일 우선 설계
-  **상태 관리**: Redux Toolkit 기반
-  **컴포넌트 중심**: 재사용 가능한 공통 컴포넌트 라이브러리
-  **멀티 탭 시스템**: 동적 탭 생성 및 관리로 향상된 UX

- 🔒 타입 안전성: ComponentType과 ReactNode 유연한 지원

🔧 기술 스택

Core Technologies

기술	버전	용도
React	18.2.0	UI 라이브러리
TypeScript	5.8.3	정적 타입 검사
Vite	5.0.12	빌드 도구 및 개발 서버
Material-UI	5.15.20	UI 컴포넌트 라이브러리
Redux Toolkit	2.8.2	상태 관리
React Router	6.26.0	클라이언트 사이드 라우팅
Tab System	Custom	멀티 탭 인터페이스

Development Tools

도구	버전	용도
ESLint	9.25.0	코드 품질 검사
Prettier	-	코드 포매팅
TypeScript ESLint	8.30.1	TypeScript 린팅
Vite React Plugin	4.6.0	React 지원

UI & Styling

라이브러리	버전	용도
@mui/material	5.15.20	기본 UI 컴포넌트
@mui/icons-material	5.15.20	아이콘
@mui/lab	5.0.0-alpha.170	실험적 컴포넌트

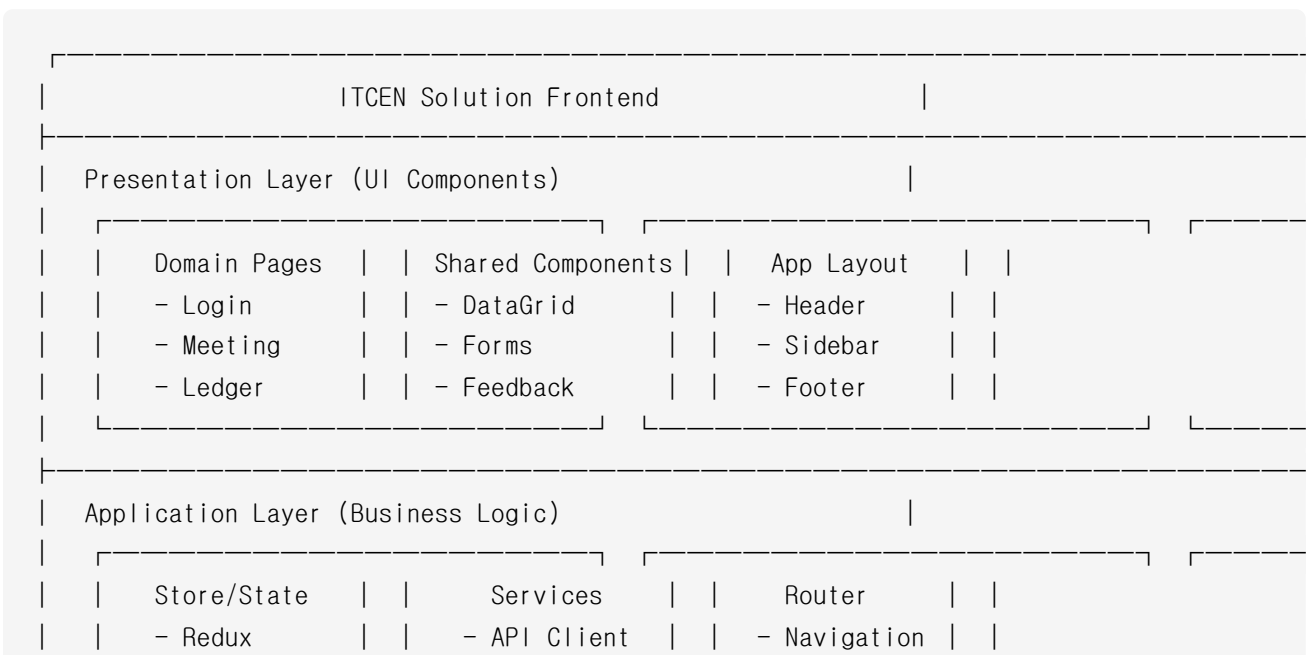
라이브러리	버전	용도
@mui/x-data-grid	7.7.1	데이터 그리드
@emotion/react	11.11.4	CSS-in-JS
@emotion/styled	11.11.5	스타일드 컴포넌트

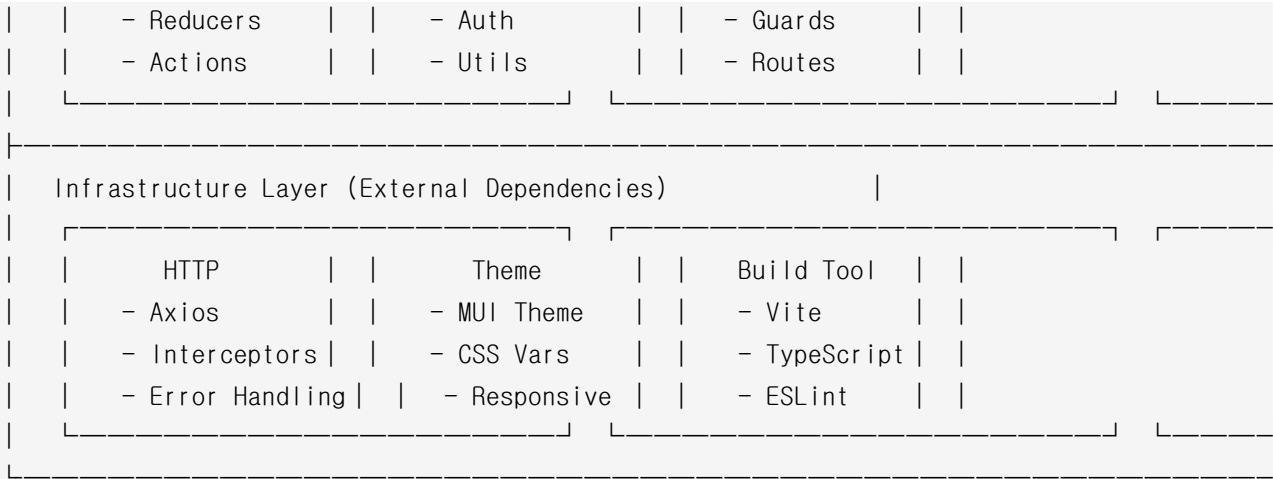
Utilities & Libraries

라이브러리	버전	용도
axios	1.9.0	HTTP 클라이언트
dayjs	1.11.13	날짜 처리
exceljs	4.4.0	Excel 파일 처리
file-saver	2.0.5	파일 다운로드
uuid	11.1.0	고유 ID 생성
cross-env	7.0.3	환경 변수 관리
rimraf	6.0.1	파일 삭제 도구

아키텍처 구조

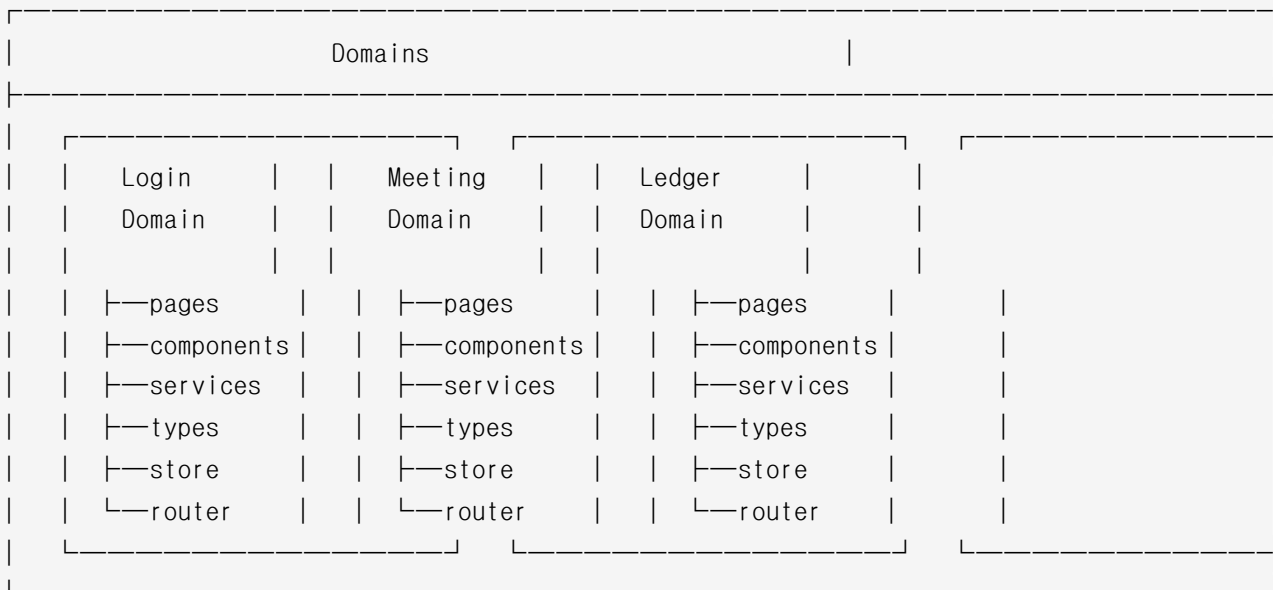
전체 아키텍처 다이어그램



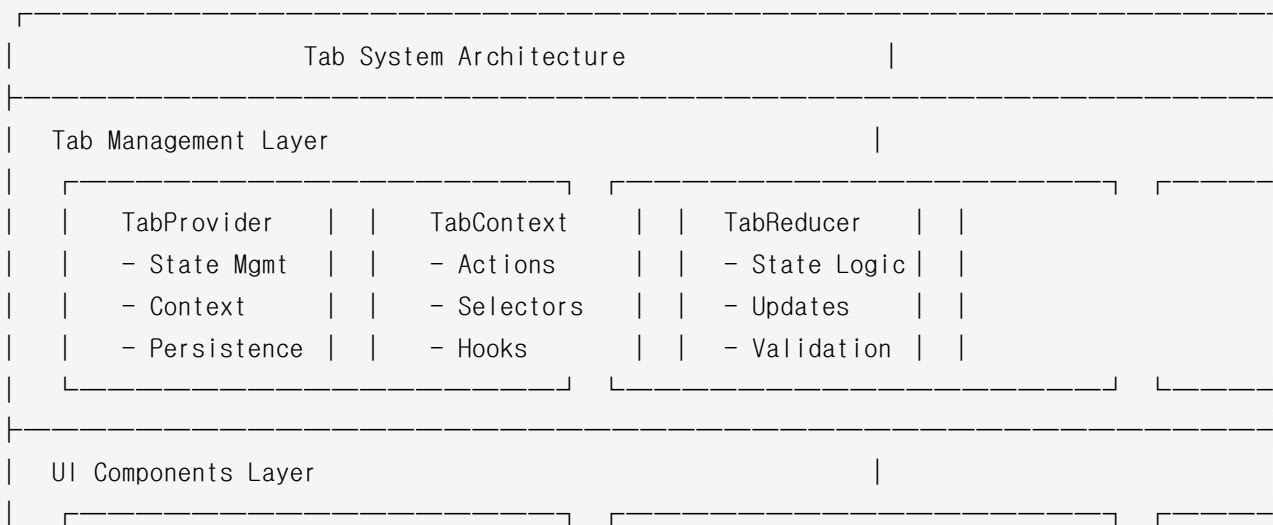


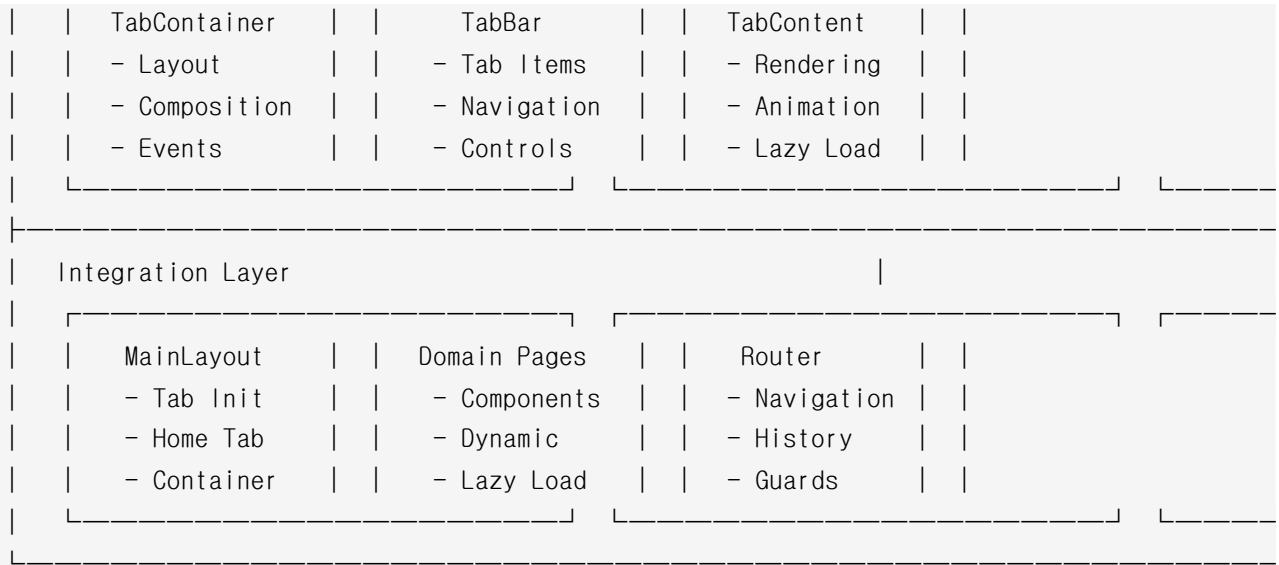
Domain-Driven Design 적용

Domain Layer Architecture:



멀티 탭 시스템 아키텍처





탭 시스템 핵심 기능:

-  **동적 탭 생성:** 런타임에 탭 추가/제거
-  **상태 관리:** Redux 기반 탭 상태 관리
-  **유연한 렌더링:** 컴포넌트 함수와 JSX 요소 모두 지원
-  **성능 최적화:** 메모이제이션 및 지연 로딩
-  **타입 안전성:** TypeScript 완전 지원

폴더 구조

전체 프로젝트 구조

```

frontend/
├── public/                                # 정적 파일
│   ├── favicon.ico
│   └── index.html
├── src/                                   # 소스 코드
│   ├── app/                              # 애플리케이션 레벨 설정
│   │   ├── components/                  # 앱 레벨 컴포넌트
│   │   ├── config/                      # 설정 파일
│   │   ├── router/                      # 라우터 설정
│   │   ├── services/                   # 글로벌 서비스
│   │   ├── store/                       # 글로벌 스토어
│   │   ├── theme/                       # 테마 설정
│   │   ├── types/                       # 글로벌 타입
│   │   └── utils/                       # 유틸리티 함수
│   ├── domains/                         # 도메인별 모듈
│   │   ├── login/                      # 로그인 도메인
│   │   ├── main/                       # 메인 페이지 도메인
│   │   ├── meeting/                    # 회의 관리 도메인
│   │   └── ledgermngt/                  # 원장 관리 도메인

```



```
|   └── index.ts           # 페이지 export
|─── services/             # 도메인 서비스
|   ├── api.ts            # API 호출
|   ├── types.ts          # API 타입
|   └── index.ts          # 서비스 export
|─── store/                # 도메인 스토어
|   ├── slice.ts          # Redux slice
|   ├── selectors.ts      # 셀렉터
|   ├── thunks.ts         # 비동기 액션
|   └── index.ts          # 스토어 export
|─── router/               # 도메인 라우터
|   ├── routes.tsx        # 라우트 정의
|   └── index.ts           # 라우터 export
|─── types/                # 도메인 타입
|   ├── entities.ts       # 엔티티 타입
|   ├── forms.ts          # 폼 타입
|   └── index.ts           # 타입 export
└─── index.ts              # 도메인 전체 export
```

개발 환경 설정

필수 요구사항

- **Node.js**: 18.0.0 이상
- **npm**: 8.0.0 이상 또는 **yarn**: 1.22.0 이상
- **Git**: 2.30.0 이상

개발 환경 구성

```
# 1. 프로젝트 클론
git clone [repository-url]
cd itcenSolution1/frontend

# 2. 의존성 설치
npm install

# 3. 환경 변수 설정
cp .env.example .env.local

# 4. 개발 서버 실행
npm run dev
```

환경 변수 설정

```
# .env.local
VITE_API_BASE_URL=http://localhost:8080
VITE_APP_TITLE=ITCEN Solution
VITE_APP_VERSION=1.0.0
VITE_APP_ENVIRONMENT=development
```

VS Code 설정

```
// .vscode/settings.json
{
  "typescript.preferences.importModuleSpecifier": "relative",
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  },
  "files.associations": {
    "*.css": "css"
  }
}
```

권장 VS Code 확장

```
// .vscode/extensions.json
{
  "recommendations": [
    "esbenp.prettier-vscode",
    "dbaeumer.vscode-eslint",
    "bradlc.vscode-tailwindcss",
    "ms-vscode.vscode-typescript-next",
    "formulahendry.auto-rename-tag",
    "christian-kohler.path-intellisense"
  ]
}
```

주요 기능별 구성

1. 인증 시스템 (Login Domain)

```
// domains/login/services/auth.ts
export interface LoginRequest {
  username: string;
  password: string;
}
```



```

}

export interface LoginResponse {
  token: string;
  user: User;
  expiresIn: number;
}

export const authService = {
  login: (credentials: LoginRequest): Promise<LoginResponse> => {
    return apiClient.post('/auth/login', credentials);
  },
  logout: (): Promise<void> => {
    return apiClient.post('/auth/logout');
  },
  refreshToken: (): Promise<string> => {
    return apiClient.post('/auth/refresh');
  },
};

```

2. 회의 관리 (Meeting Domain)

```

// domains/meeting/types/entities.ts
export interface Meeting {
  id: string;
  title: string;
  date: Date;
  participants: Participant[];
  agenda: AgendaItem[];
  status: MeetingStatus;
}

export interface AgendaItem {
  id: string;
  title: string;
  description: string;
  duration: number;
  presenter: string;
}

```

3. 장부 관리 (Ledger Management Domain)

```

// domains/ledgermngt/services/ledger.ts
export interface LedgerEntry {
  id: string;
  date: Date;
  description: string;
}

```

```

    amount: number;
    category: string;
    type: 'income' | 'expense';
  }

export const ledgerService = {
  getEntries: (params: LedgerSearchParams): Promise<LedgerEntry[]> => {
    return apiClient.get('/ledger/entries', { params });
  },
  createEntry: (entry: CreateLedgerEntry): Promise<LedgerEntry> => {
    return apiClient.post('/ledger/entries', entry);
  },
};

```

4. 멀티 탭 시스템 (Tab System)

```

// shared/context/TabContext.tsx
export interface Tab {
  id: string;
  title: string;
  path: string;
  component: ComponentType<any> | ReactNode;
  closable?: boolean;
  icon?: string;
  createdAt: Date;
  lastActiveAt: Date;
}

export interface TabContextType {
  state: TabManagerState;
  tabs: Tab[];
  activeTabId: string | null;
  addTab: (tab: Omit<Tab, 'id' | 'createdAt' | 'lastActiveAt'>) => void;
  removeTab: (tabId: string) => void;
  setActiveTab: (tabId: string) => void;
}

// 탭 시스템 사용 예시
export const useTabSystem = () => {
  const { addTab, removeTab, setActiveTab, tabs } = useTabContext();

  const openNewTab = useCallback(
    (page: ComponentType, title: string, path: string) => {
      addTab({
        title,
        path,
        component: page,
        closable: true,

```

```

    });
  },
  [addTab]
);

return { openNewTab, tabs };
};

```

5. 컴포넌트 렌더링 시스템

```

// shared/components/tabs/TabContent.tsx
const renderTabComponent = (component: ComponentType<any> | React.ReactNode) => {
  // 이미 렌더링된 React 요소인지 확인
  if (isValidElement(component)) {
    return component;
  }

  // 컴포넌트 함수인지 확인
  if (typeof component === 'function') {
    const Component = component as ComponentType<any>;
    return <Component />;
  }

  // 그 외의 경우 직접 렌더링
  return component;
};

```



상태 관리

Redux Toolkit 구조

```

// app/store/index.ts
import { configureStore } from '@reduxjs/toolkit';
import { loginSlice } from '@domains/login/store/slice';
import { meetingSlice } from '@domains/meeting/store/slice';
import { ledgerSlice } from '@domains/ledgermngt/store/slice';

export const store = configureStore({
  reducer: {
    login: loginSlice.reducer,
    meeting: meetingSlice.reducer,
    ledger: ledgerSlice.reducer,
  },
  middleware: getDefaultMiddleware =>
    getDefaultMiddleware({

```

```

    serializableCheck: {
      ignoredActions: ['persist/PERSIST'],
    },
  })),
});

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;

```

도메인별 Slice 예시

```

// domains/meeting/store/slice.ts
import { createSlice, PayloadAction } from '@reduxjs/toolkit';
import { Meeting } from '../../types/entities';

interface MeetingState {
  meetings: Meeting[];
  selectedMeeting: Meeting | null;
  loading: boolean;
  error: string | null;
}

const initialState: MeetingState = {
  meetings: [],
  selectedMeeting: null,
  loading: false,
  error: null,
};

export const meetingSlice = createSlice({
  name: 'meeting',
  initialState,
  reducers: {
    setMeetings: (state, action: PayloadAction<Meeting[]>) => {
      state.meetings = action.payload;
    },
    setSelectedMeeting: (state, action: PayloadAction<Meeting>) => {
      state.selectedMeeting = action.payload;
    },
    setLoading: (state, action: PayloadAction<boolean>) => {
      state.loading = action.payload;
    },
    setError: (state, action: PayloadAction<string | null>) => {
      state.error = action.payload;
    },
  },
});

```

커스텀 훅 활용

```
// shared/hooks/useAppSelector.ts
import { useSelector, TypedUseSelectorHook } from 'react-redux';
import type { RootState } from '@app/store';

export const useAppSelector: TypedUseSelectorHook<RootState> = useSelector;

// domains/meeting/hooks/useMeeting.ts
export const useMeeting = () => {
  const dispatch = useAppDispatch();
  const { meetings, selectedMeeting, loading, error } = useAppSelector(state => state.meetings);

  const fetchMeetings = useCallback(() => {
    dispatch(fetchMeetingsThunk());
  }, [dispatch]);

  return {
    meetings,
    selectedMeeting,
    loading,
    error,
    fetchMeetings,
  };
};
```

라우팅 시스템

라우터 구조

```
// app/router/AppRouter.tsx
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import { LoginRoutes } from '@domains/login/router';
import { MeetingRoutes } from '@domains/meeting/router';
import { LedgerRoutes } from '@domains/ledgermngt/router';

const router = createBrowserRouter([
  {
    path: '/',
    element: <AppLayout />,
    children: [...LoginRoutes, ...MeetingRoutes, ...LedgerRoutes],
  },
]);

export const AppRouter: React.FC = () => {
```

```
return <RouterProvider router={router} />;  
};
```

도메인별 라우트 정의

```
// domains/meeting/router/routes.tsx  
import { RouteObject } from 'react-router-dom';  
import { MeetingListPage } from '../../pages/MeetingListPage';  
import { MeetingDetailPage } from '../../pages/MeetingDetailPage';  
  
export const MeetingRoutes: RouteObject[] = [  
  {  
    path: '/meetings',  
    children: [  
      {  
        index: true,  
        element: <MeetingListPage />,  
      },  
      {  
        path: ':id',  
        element: <MeetingDetailPage />,  
      },  
      {  
        path: 'create',  
        element: <MeetingCreatePage />,  
      },  
    ],  
  },  
];
```

라우터 유틸리티

```
// shared/router/RouterUtil.ts  
export class RouterUtil {  
  private static navigate: NavigateFunction | null = null;  
  
  public static setNavigate(navigate: NavigateFunction): void {  
    this.navigate = navigate;  
  }  
  
  public static push(path: string, options?: NavigateOptions): void {  
    if (!this.navigate) {  
      console.error('[Router] Navigate instance not set');  
      return;  
    }  
    this.navigate(path, options);  
  }  
}
```

```
public static goBack(): void {
  if (!this.navigate) return;
  this.navigate(-1);
}

public static getCurrentPath(): string {
  return window.location.pathname;
}
}
```

스타일링 시스템

Material-UI 테마 설정

```
// app/theme/index.ts
import { createTheme } from '@mui/material/styles';

export const theme = createTheme({
  palette: {
    primary: {
      main: '#1976d2',
      light: '#42a5f5',
      dark: '#1565c0',
    },
    secondary: {
      main: '#dc004e',
      light: '#ff5983',
      dark: '#9a0036',
    },
  },
  typography: {
    fontFamily: [
      'Roboto',
      '-apple-system',
      'BlinkMacSystemFont',
      '"Segoe UI"',
      'Arial',
      'sans-serif',
    ].join(','),
    button: {
      textTransform: 'none',
    },
  },
  shape: {
    borderRadius: 8,
  },
});
```

```

components: {
  MuiButton: {
    styleOverrides: {
      root: {
        borderRadius: 8,
        fontWeight: 500,
      },
    },
  },
  MuiCard: {
    styleOverrides: {
      root: {
        borderRadius: 12,
        boxShadow: '0px 2px 8px rgba(0, 0, 0, 0.1)',
      },
    },
  },
},
});

```

CSS Variables 활용

```

// app/theme/cssVariables.ts
export const cssVariables = {
  '--primary-color': '#1976d2',
  '--secondary-color': '#dc004e',
  '--success-color': '#2e7d32',
  '--warning-color': '#ed6c02',
  '--error-color': '#d32f2f',
  '--info-color': '#0288d1',
  '--border-radius': '8px',
  '--box-shadow': '0px 2px 8px rgba(0, 0, 0, 0.1)',
};

```

반응형 디자인

```

// shared/utils/responsive.ts
export const breakpoints = {
  xs: '0px',
  sm: '600px',
  md: '900px',
  lg: '1200px',
  xl: '1536px',
};

export const useResponsive = () => {
  const theme = useTheme();

```



```

return {
  isMobile: useMediaQuery(theme.breakpoints.down('sm')),
  isTablet: useMediaQuery(theme.breakpoints.between('sm', 'lg')),
  isDesktop: useMediaQuery(theme.breakpoints.up('lg')),
};
};

```

✖ 공통 컴포넌트

컴포넌트 라이브러리 구조

```

shared/components/ui/
├── button/
│   ├── Button.tsx
│   └── index.ts
├── form/
│   ├── Select.tsx
│   ├── ComboBox.tsx
│   ├── DatePicker.tsx
│   ├── FileUpload.tsx
│   ├── ServerFileUpload.tsx
│   └── index.ts
├── data-display/
│   ├── DataGrid.tsx
│   ├── ServerDataGrid.tsx
│   ├── Chip.tsx
│   ├── Badge.tsx
│   └── index.ts
├── feedback/
│   ├── Toast.tsx
│   ├── ToastProvider.tsx
│   ├── Loading.tsx
│   ├── LoadingProvider.tsx
│   ├── Alert.tsx
│   ├── Modal.tsx
│   └── index.ts
├── layout/
│   ├── Container.tsx
│   ├── Grid.tsx
│   ├── Stack.tsx
│   └── index.ts
└── index.ts

```

```

shared/components/tabs/
├── TabBar.tsx      # 탭 바 컴포넌트
├── TabContainer.tsx # 탭 컨테이너

```

```
|—— TabContent.tsx      # 탭 콘텐츠 렌더링
└—— index.ts           # 탭 시스템 export
```

주요 컴포넌트 예시

```
// shared/components/ui/data-display/DataGrid.tsx
export interface DataGridProps<T = any> {
  data: T[];
  columns: DataGridColumn<T>[];
  loading?: boolean;
  autoHeight?: boolean;
  pagination?: PaginationProps;
  onRowClick?: (row: T) => void;
  onRowSelectionChange?: (selectedRows: T[]) => void;
}

export const DataGrid = <T extends Record<string, any>>({
  data,
  columns,
  autoHeight = true,
  ...props
}: DataGridProps<T>) => {
  return <MuiDataGrid rows={data} columns={columns} autoHeight={autoHeight} {...props} />;
};
```

글로벌 상태 관리 컴포넌트

```
// shared/components/ui/feedback/ToastProvider.tsx
export const ToastProvider: React.FC<{ children: React.ReactNode }> = ({ children }) => {
  const [toasts, setToasts] = useState<Toast[]>([]);

  const showToast = useCallback((message: string, severity: ToastSeverity) => {
    const newToast: Toast = {
      id: uuidv4(),
      message,
      severity,
      timestamp: Date.now(),
    };
    setToasts(prev => [...prev, newToast]);
  }, []);

  return (
    <ToastContext.Provider value={{ showToast, hideToast }}>
      {children}
      <ToastContainer toasts={toasts} />
    </ToastContext.Provider>
  );
};
```

```
);  
};
```

멀티 탭 시스템

탭 시스템 개요

ITCEN Solution의 멀티 탭 시스템은 사용자가 여러 페이지를 동시에 열고 관리할 수 있는 고급 사용자 인터페이스를 제공합니다. 이 시스템은 Context API와 useReducer를 활용한 상태 관리, 동적 컴포넌트 렌더링, 그리고 TypeScript 타입 안전성을 보장합니다.

핵심 컴포넌트 구조

```
// TabContext - 상태 관리 중심  
export const TabProvider: React.FC<TabProviderProps> = ({ children }) => {  
  const [state, dispatch] = useReducer(tabReducer, initialState);  
  
  const contextValue: TabContextType = {  
    state,  
    tabs: state.tabs,  
    activeTabId: state.activeTabId,  
    addTab: tab => dispatch({ type: 'ADD_TAB', payload: tab }),  
    removeTab: tabId => dispatch({ type: 'REMOVE_TAB', payload: { tabId } }),  
    setActiveTab: tabId => dispatch({ type: 'SET_ACTIVE_TAB', payload: { tabId } }),  
  };  
  
  return <TabContext.Provider value={contextValue}>{children}</TabContext.Provider>;  
};  
  
// TabContainer - 레이아웃 관리  
export const TabContainer: React.FC<TabContainerProps> = ({  
  showTabBar = true,  
  maxTabWidth = 200,  
}) => {  
  const { tabs, activeTabId, setActiveTab, removeTab } = useTabContext();  
  
  return (  
    <Box sx={{ display: 'flex', flexDirection: 'column', height: '100%' }}>  
      {showTabBar && (  
        <TabBar  
          tabs={tabs}  
          activeTabId={activeTabId}  
          onTabClick={setActiveTab}  
          onTabClose={removeTab}  
          maxTabWidth={maxTabWidth}  
        />  
      )}
```

```

    })
    <TabContent activeTab={tabs.find(tab => tab.id === activeTabId) || null} />
  </Box>
);
};

```

동적 컴포넌트 렌더링

```

// TabContent - 유연한 컴포넌트 렌더링
const renderTabComponent = (component: ComponentType<any> | React.ReactNode) => {
  // React 요소인지 확인
  if (isValidElement(component)) {
    return component;
  }

  // 컴포넌트 함수인지 확인
  if (typeof component === 'function') {
    const Component = component as ComponentType<any>;
    return <Component />;
  }

  // 기타 ReactNode (문자열, 숫자 등)
  return component;
};

export const TabContent: React.FC<TabContentProps> = ({ activeTab }) => {
  if (!activeTab) {
    return <EmptyTabMessage />;
  }

  return (
    <Suspense fallback={<LoadingSpinner />}>
      <Box sx={{ flex: 1, overflow: 'hidden' }}>{renderTabComponent(activeTab.component)}</Box>
    </Suspense>
  );
};

```

탭 생성 및 관리

```

// MainLayout에서 홈 탭 자동 생성
const HomeTabInitializer: React.FC = () => {
  const { addTab, tabs = [] } = useTabContext();

  useEffect(() => {
    const homeTabExists = Array.isArray(tabs) && tabs.some(tab => tab.path === '/main');

    if (!homeTabExists) {

```

```

    addTab({
      title: '홈',
      path: '/main',
      component: MainPage, // 컴포넌트 함수 전달
      closable: false,
      icon: 'home',
    });
  }
}, [addTab, tabs]);

return null;
};

// 도메인에서 새 탭 생성
const useNavigateTab = () => {
  const { addTab, setActiveTab } = useTabContext();

  const navigateToTab = useCallback(
    (title: string, component: ComponentType<any>, path: string) => {
      const newTab = {
        title,
        path,
        component,
        closable: true,
      };

      addTab(newTab);
      // 탭이 추가되면 자동으로 활성화됨
    },
    [addTab]
  );

  return { navigateToTab };
};

```

상태 관리 아키텍처

```

// Tab Reducer - 상태 변화 로직
const tabReducer = (state: TabManagerState, action: TabAction): TabManagerState => {
  switch (action.type) {
    case 'ADD_TAB': {
      // 중복 탭 확인
      const existingTab = state.tabs.find(tab => tab.path === action.payload.path);
      if (existingTab) {
        return {
          ...state,
          activeTabId: existingTab.id,
          tabs: state.tabs.map(tab =>

```

```

        tab.id === existingTab.id ? { ...tab, lastActiveAt: new Date() } : tab
    ),
    };
}

// 새 탭 생성
const newTab: Tab = {
    ...action.payload,
    id: action.payload.path === '/main' ? 'home' : uuidv4(),
    createdAt: new Date(),
    lastActiveAt: new Date(),
};

return {
    ...state,
    tabs: [...state.tabs, newTab],
    activeTabId: newTab.id,
};
}

case 'REMOVE_TAB': {
    const newTabs = state.tabs.filter(tab => tab.id !== action.payload.tabId);
    let newActiveTabId = state.activeTabId;

    // 활성 탭이 제거되면 홈 탭으로 이동
    if (state.activeTabId === action.payload.tabId) {
        const homeTab = newTabs.find(tab => tab.id === 'home');
        newActiveTabId = homeTab ? homeTab.id : newTabs[0]?.id || null;
    }

    return {
        ...state,
        tabs: newTabs,
        activeTabId: newActiveTabId,
    };
}

default:
    return state;
}
};

```

타입 안전성

```

// 타입 정의
export interface Tab {
    id: string;
    title: string;
}

```

```

    path: string;
    component: ComponentType<any> | ReactNode; // 유연한 컴포넌트 타입
    closable?: boolean;
    icon?: string;
    params?: Record<string, unknown>;
    state?: Record<string, unknown>;
    isDirty?: boolean;
    createdAt: Date;
    lastActiveAt: Date;
  }

export interface TabManagerState {
  tabs: Tab[];
  activeTabId: string | null;
  maxTabs: number;
  homeTabId: string;
}

// Hook 타입 안전성
export const useTabContext = (): TabContextType => {
  const context = useContext(TabContext);
  if (!context) {
    throw new Error('useTabContext must be used within a TabProvider');
  }
  return context;
};

```

성능 최적화

```

// 메모이제이션 활용
export const TabBar = React.memo<TabBarProps>(
  ({ tabs, activeTabId, onTabClick, onTabClose, maxTabWidth }) => {
    const memoizedTabs = useMemo(
      () =>
        tabs.map(tab => ({
          ...tab,
          isActive: tab.id === activeTabId,
        })),
      [tabs, activeTabId]
    );

    return (
      <Box sx={{ display: 'flex', borderBottom: 1, borderColor: 'divider' }}>
        {memoizedTabs.map(tab => (
          <TabItem
            key={tab.id}
            tab={tab}
            isActive={tab.isActive}

```

```

        onTabClick={onTabClick}
        onTabClose={onTabClose}
        maxWidth={maxTabWidth}
      />
    )}}
  </Box>
);
}
);

// 지연 로딩
const LazyTabContent = lazy(() => import('./TabContent'));

// 사용 시
<Suspense fallback={<TabLoadingSkeleton />}>
  <LazyTabContent activeTab={activeTab} />
</Suspense>;

```

빌드 및 배포

Vite 빌드 설정

```

// vite.config.ts
export default defineConfig(({ mode }) => {
  return {
    plugins: [react()],
    resolve: {
      alias: {
        '@': path.resolve(__dirname, './src'),
      },
    },
    build: {
      outDir: 'dist',
      sourcemap: mode === 'development',
      minify: mode === 'production',
      rollupOptions: {
        output: {
          manualChunks: {
            vendor: ['react', 'react-dom'],
            mui: ['@mui/material', '@mui/icons-material'],
            utils: ['axios', 'dayjs'],
          },
        },
      },
    },
    server: {
      host: '0.0.0.0',
    },
  };
});

```



```

    port: 3000,
    proxy: {
      '/api': {
        target: process.env.VITE_API_BASE_URL || 'http://localhost:8080',
        changeOrigin: true,
        secure: false,
      },
    },
  },
};
});

```

Docker 설정

```

# Dockerfile
FROM node:18-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]

```

Nginx 설정

```

# nginx.conf
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    # Gzip 압축
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml applica

    # 정적 파일 캐싱
    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 1y;
    }
}

```

```

    add_header Cache-Control "public, immutable";
}

# SPA 라우팅
location / {
    try_files $uri $uri/ /index.html;
}

# API 프록시
location /api/ {
    proxy_pass http://backend:8080/;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
}

```

빌드 스크립트

```

{
  "scripts": {
    "dev": "vite",
    "build": "cross-env NODE_OPTIONS=W\"--max-old-space-size=4096W\" tsc -b && vite build",
    "build:clean": "rimraf dist && npm run build",
    "preview": "vite preview",
    "lint": "eslint .",
    "lint:fix": "eslint . --fix",
    "type-check": "tsc --noEmit"
  }
}

```

개발 가이드라인

코딩 컨벤션

TypeScript 규칙

```

// ✅ 좋은 예시
interface UserProps {
  id: string;
  name: string;
  email: string;
  isActive?: boolean;
}

const UserComponent: React.FC<UserProps> = ({ id, name, email, isActive = true }) => {

```

```

const [loading, setLoading] = useState<boolean>(false);

const handleSubmit = useCallback(
  async (data: FormData) => {
    setLoading(true);
    try {
      await userService.updateUser(id, data);
    } catch (error) {
      console.error('Failed to update user:', error);
    } finally {
      setLoading(false);
    }
  },
  [id]
);

return (
  <Card>
    <Typography variant='h6'>{name}</Typography>
    <Typography variant='body2'>{email}</Typography>
  </Card>
);
};

```

컴포넌트 구조

```

// 컴포넌트 파일 구조
import React, { useState, useCallback, useMemo } from 'react';
import { Box, Typography, Button } from '@mui/material';
import { useAppDispatch, useAppSelector } from '@shared/hooks';
import { ComponentProps } from '../types';

// 1. 인터페이스 정의
interface MyComponentProps extends ComponentProps {
  title: string;
  onSubmit: (data: FormData) => void;
}

// 2. 컴포넌트 정의
export const MyComponent: React.FC<MyComponentProps> = ({ title, onSubmit, ...props }) => {
  // 3. 상태 관리
  const [state, setState] = useState<ComponentState>({});

  // 4. 훅 사용
  const dispatch = useAppDispatch();
  const data = useAppSelector(selectData);

  // 5. 메모이제이션

```

```

const memoizedValue = useMemo(() => {
  return computeExpensiveValue(data);
}, [data]);

// 6. 이벤트 핸들러
const handleClick = useCallback(() => {
  // 핸들러 로직
}, []);

// 7. 렌더링
return (
  <Box {...props}>
    <Typography variant='h6'>{title}</Typography>
    <Button onClick={handleClick}>Submit</Button>
  </Box>
);
};

export default MyComponent;

```

파일 명명 규칙

컴포넌트 파일:	PascalCase.tsx (UserCard.tsx)
페이지 파일:	PascalCase.tsx (UserListPage.tsx)
훅 파일:	camelCase.ts (useUserData.ts)
유틸리티 파일:	camelCase.ts (formatDate.ts)
타입 파일:	camelCase.ts (userTypes.ts)
상수 파일:	camelCase.ts (apiConstants.ts)

Import 순서

```

// 1. React 관련
import React, { useState, useEffect } from 'react';

// 2. 외부 라이브러리
import { Box, Typography } from '@mui/material';
import { useDispatch } from 'react-redux';

// 3. 내부 모듈 (절대 경로)
import { useAppSelector } from '@shared/hooks';
import { Button } from '@shared/components/ui';

// 4. 상대 경로 import
import { ComponentProps } from '../types';
import { useLocalHook } from '../hooks/useLocalHook';

```

성능 최적화

코드 스플리팅

```
// 라우트 레벨 코드 스플리팅
const LazyUserListPage = lazy(() => import('@domains/user/pages/UserListPage'));
const LazyUserDetailPage = lazy(() => import('@domains/user/pages/UserDetailPage'));

// 컴포넌트 레벨 코드 스플리팅
const LazyDataGrid = lazy(() => import('@shared/components/ui/DataGrid'));

// 사용 예시
const UserRoutes = [
  {
    path: '/users',
    element: (
      <Suspense fallback={<Loading />}>
        <LazyUserListPage />
      </Suspense>
    ),
  },
];
```

메모이제이션 최적화

```
// React.memo 사용
export const UserCard = React.memo<UserCardProps>(({ user, onEdit }) => {
  return (
    <Card>
      <Typography>{user.name}</Typography>
      <Button onClick={() => onEdit(user.id)}>Edit</Button>
    </Card>
  );
});

// useMemo 사용
const ExpensiveComponent: React.FC = ({ data }) => {
  const processedData = useMemo(() => {
    return data.map(item => ({
      ...item,
      computed: expensiveComputation(item),
    }));
  }, [data]);

  return <DataGrid data={processedData} />;
};
```

```
// useCallback 사용
const ParentComponent: React.FC = () => {
  const handleUserEdit = useCallback((userId: string) => {
    // 편집 로직
  }, []);

  return <UserList onUserEdit={handleUserEdit} />;
};
```

번들 크기 최적화

```
// Tree shaking을 위한 named import 사용
import { Button, TextField } from '@mui/material';

// 전체 import 지양
// import * as MUI from '@mui/material'; ❌

// 동적 import 활용
const importHeavyLibrary = async () => {
  const { heavyFunction } = await import('heavy-library');
  return heavyFunction;
};
```

이미지 최적화

```
// 이미지 lazy loading
const OptimizedImage: React.FC<ImageProps> = ({ src, alt }) => {
  return (
    <img
      src={src}
      alt={alt}
      loading='lazy'
      decoding='async'
      style={{ maxWidth: '100%', height: 'auto' }}
    />
  );
};

// WebP 지원 확인
const ImageWithFallback: React.FC<ImageProps> = ({ src, alt }) => {
  const [imageSrc, setImageSrc] = useState(src);

  const handleError = () => {
    setImageSrc(src.replace('.webp', '.jpg'));
  };
};
```

```
return <img src={imageSrc} alt={alt} onError={handleError} loading='lazy' />;
};
```

트러블슈팅

자주 발생하는 문제들

1. DataGrid 높이 문제

문제: MUI X DataGrid v7.7.1에서 높이 관련 오류

MUI X: useResizeContainer - The parent DOM element of the Data Grid has an empty height.

해결책:

```
// ✅ autoHeight 속성 사용
<DataGrid
  data={data}
  columns={columns}
  autoHeight // 자동 높이 조정
/>

// ✅ 고정 높이 설정
<DataGrid
  data={data}
  columns={columns}
  height={600}
/>
```

2. TypeScript 경로 별칭 문제

문제: @/ 경로 별칭이 인식되지 않음

해결책:

```
// tsconfig.app.json
{
  "compilerOptions": {
    "paths": {
      "@/*": ["../src/*"]
    }
  }
}
```

```
// vite.config.ts
export default defineConfig({
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
});
```

3. 빌드 메모리 부족 문제

문제: 빌드 시 메모리 부족 오류

해결책:

```
// package.json
{
  "scripts": {
    "build": "cross-env NODE_OPTIONS=W"--max-old-space-size=4096W" tsc -b && vite build"
  }
}
```

4. CORS 문제

문제: 개발 환경에서 API 호출 시 CORS 오류

해결책:

```
// vite.config.ts
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:8080',
        changeOrigin: true,
        secure: false,
      },
    },
  },
});
```

5. 탭 시스템 렌더링 문제

문제: “Functions are not valid as a React child” 에러

원인: 컴포넌트 함수를 JSX 요소로 직접 렌더링하려고 시도

해결책:

```
// ✅ 올바른 해결 방법
const renderTabComponent = (component: ComponentType<any> | React.ReactNode) => {
  if (isValidElement(component)) {
    return component;
  }

  if (typeof component === 'function') {
    const Component = component as ComponentType<any>;
    return <Component />;
  }

  return component;
};

// ❌ 잘못된 방법
{
  activeTab.component;
} // 컴포넌트 함수 직접 렌더링

// ✅ 올바른 방법
{
  renderTabComponent(activeTab.component);
}
```

6. useTabContext 정의되지 않음 문제

문제: “Cannot read properties of undefined (reading ‘some’)” 에러

원인: TabContextType에서 tabs 배열이 직접 노출되지 않음

해결책:

```
// TabContextType 인터페이스 개선
export interface TabContextType {
  state: TabManagerState;
  tabs: Tab[]; // 직접 접근을 위한 tabs 배열 노출
  activeTabId: string | null;
  // ... 기타 메서드들
}

// 방어 코드 추가
const { addTab, tabs = [] } = useTabContext(); // 기본값 설정
const homeTabExists = Array.isArray(tabs) && tabs.some(tab => tab.path === '/main');
```

디버깅 도구

React Developer Tools

```
// 개발 환경에서만 활성화
if (process.env.NODE_ENV === 'development') {
  // React Developer Tools 확장 설치 권장
  console.log('React Developer Tools를 설치하여 컴포넌트를 디버깅하세요.');
```

Redux DevTools

```
// store 설정에서 Redux DevTools 활성화
export const store = configureStore({
  reducer: rootReducer,
  devTools: process.env.NODE_ENV !== 'production',
});
```

성능 모니터링

```
// 성능 측정
const PerformanceMonitor: React.FC = ({ children }) => {
  useEffect(() => {
    if (process.env.NODE_ENV === 'development') {
      const observer = new PerformanceObserver(list => {
        list.getEntries().forEach(entry => {
          console.log(`${entry.name}: ${entry.duration}ms`);
        });
      });

      observer.observe({ entryTypes: ['measure'] });

      return () => observer.disconnect();
    }
  }, []);

  return <>{children}</>;
};
```

참고 자료

공식 문서

- [React 18 Documentation](#)
- [TypeScript Handbook](#)

- [Vite Guide](#)
- [Material-UI Documentation](#)
- [Redux Toolkit Documentation](#)

내부 문서

- [공통 컴포넌트 가이드](#)
- [컴포넌트 사용 예시](#)
- [프로젝트 README](#)

개발 도구

- [VS Code](#)
- [React Developer Tools](#)
- [Redux DevTools](#)

 변경 이력

버전	날짜	변경 내용
1.0.0	2024.12	초기 문서 작성
1.1.0	2024.12	DataGrid autoHeight 문제 해결 추가
1.2.0	2024.12	글로벌 Toast/Loading 시스템 추가
1.3.0	2024.12	서버 통합 컴포넌트 추가
1.4.0	2025.01	멀티 탭 시스템 아키텍처 추가
1.5.0	2025.01	컴포넌트 렌더링 시스템 개선
1.6.0	2025.01	기술 스택 버전 정보 현행화

 최신 업데이트 (2025.01)

 새로운 아키텍처

- **멀티 탭 시스템:** 동적 탭 생성 및 관리 아키텍처
- **컴포넌트 렌더링:** 유연한 컴포넌트 렌더링 시스템
- **타입 안전성:** ComponentType과 ReactNode 유연한 지원

기술적 개선사항

- **에러 처리:** 탭 시스템 관련 에러 핸들링 가이드
- **성능 최적화:** 탭 메모리 관리 및 렌더링 최적화
- **개발자 경험:** TypeScript 타입 안전성 강화

문서 개선

- **아키텍처 다이어그램:** 탭 시스템 구조 시각화
- **구현 가이드:** 실제 코드 예시 및 사용법
- **트러블슈팅:** 최근 해결된 이슈들 문서화

문서 작성자: ITCEN Solution 개발팀

최종 업데이트: 2025년 1월

문서 버전: 1.6.0